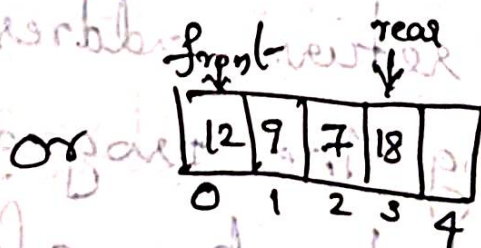
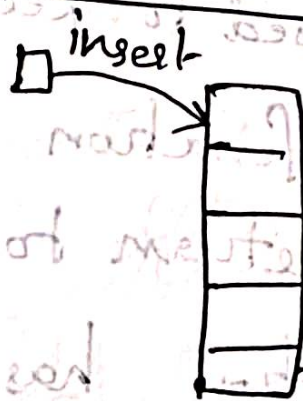


Queue

→ is a data structure - that first item inserted is the first item to be removed (FIFO).

Queue Representation:



Basic Operations:

* insert/enqueue - add an item to the rear.

* remove/dequeue - remove an item from the front

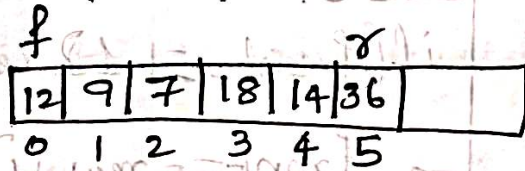
Few more Operations:

peek - get the element at front of the queue

is full - check if queue is full.

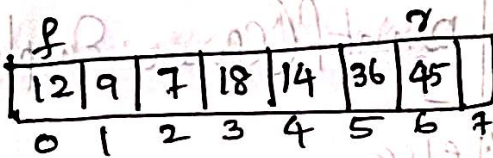
is empty - check if queue is empty.

Insert/Enqueue Operations.



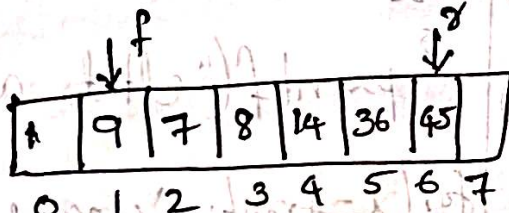
Initially Queue:

f = 0, r = 5



After inserting (45) at end

f = 0, r = 6



After deleting 12 from front.

f = 1, r = 6

Algorithm: INSERT

step 1: start

2: if rear = Max - 1

write overflow

go to step 5.

3: if (front == -1 & rear == -1)

set front = rear = 0

else

set rear = rear + 1

4: set queue[rear] = number;

5: Exit.

Algorithm: DELETE

step 1: if front == -1 or

front > rear

write overflow.

2: else

set val = queue[front]

set front = front + 1

3: Exit.

Programs:

main c)

```
{ int ch, front = -1, rear = -1,
  q[10], items, i;
```

```
while(1)
```

```
{ printf("Queue 1: insert 2. delete  
3: display 4: exit");
```

```
scanf("%d", &ch);
```

```
switch (ch)
```

```
{ case 1: if (rear == 9)
  { printf("Queue full");
```

```
else
```

```
{ if (front == -1 & rear == -1)
```

```
{ front = 0; rear = 0;
```

```
}
```

```
else
```

```
rear ++;
```

```
printf("Enter the item
```

```
to be inserted");
```

```
scanf("%d", &items);
```

```
q[rear] = items;
```

```
}
```

```
case 2:
```

```
if (front == -1 & rear == -1)
```

```
{ front = rear + 1;
  printf("Queue empty");
```

```
}
```

```
else
```

```
{ item = q[front];
```

```
front ++;
```

```
printf("deleted item %d", item);
```

```
} break;
```

```
case 3:
```

```
if (front == -1 & rear == -1)
```

```
{ front = rear + 1;
```

```
printf("Queue Empty");
```

```
else
```

```
{ printf("Elements are");
```

```
for (i = front; i <= rear; i++)
```

```
printf("%d ", q[i]);
```

```
}
```

```
break;
```

```
case 4: exit(0);
```

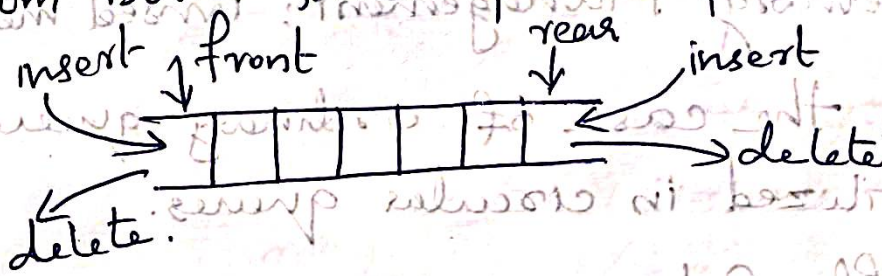
```
break;
```

```
}
```

Classification of Queues

→ Double Ended Queue (Deque)

* → insertion & deletion ops are performed at both the ends (front & rear). We can insert at the front end & rear positions & can delete from both front & rear positions.



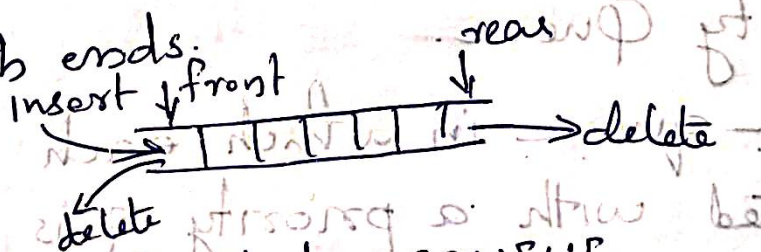
DEQUEUE can be represented in 2 ways:

1. Input restricted DEQUEUE
2. O/p restricted DEQUEUE

Input Restricted DEQUEUE

The insertion operation is performed at only ~~one~~ one end & deletion opⁿ is performed

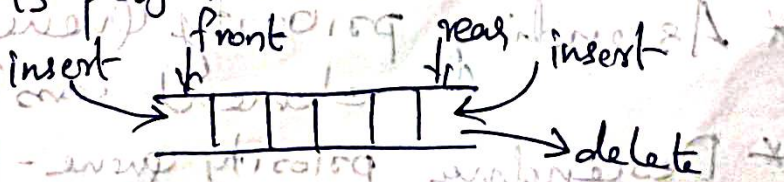
at both ends.



Output Restricted DEQUEUE

Deletion is performed at only one end &

insertion is performed at both the ends.



→ Circular Queue:

→ Based on FIFO principle & the last position is connected back to the first position to make a circle. It is also called Ring Buffer.

used in:

→ Memory Management: unused memory locations in the case of ordinary queues can be utilized in circular queues.

→ Traffic System: Computer Controlled traffic s/m - switch on the traffic lights one by one repeatedly as per the time set.

→ CPU Scheduling: O-S maintains a queue of processes that are ready to execute or that are waiting for a particular event to occur.

→ Priority Queue.

- queue in which each element is associated with a priority & is served according to its priority.

2 types.

* Ascending priority Queue - only smallest element can be removed.

* Descending priority Queue - largest element can be removed.

Applications of Queue Data Structure

- Task Scheduling - schedule based on priority or the order.
- Resource Allocation - manage allocation of resources.
- Message Buffering - Communication spms.
- Event Handling - GUI Applications
- Traffic Management - traffic flow in transportation - airport controls/m & road.
- Printer Queues - order in which printer processes.
- Web Servers - requests from clients.